A Mini Project Abstract

On

"Virtual Security Operation Center (VSOC)"

Submitted to CMR ENGINEERING COLLEGE

In Partial Fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY
IN
CSE- CYBER SECURITY

Submitted
By

Y. RAGHAVA	**228R1A6264**
P. RAJEEV	**228R1A6251**
K. MODITHA	**228R1A6236**
M. AJAY	**228R1A6243**

Under the guidance of

Ms. T.Usha

Assistant Professor, Department of CSE-Cyber Security



Department CSE-Cyber Security

CMR ENGINEERING COLLEGE UGC AUTONOMOUS

(Accredited by NBA Approved by AICTE, NEW DELHI, Affiliated to JNTU Hyderabad) Kandlakoya, Medchal Road, Medchal, Malkajgiri Dist. Hyderabad-501-401)

2024-2025

CMR ENGINEERING COLLEGE

UGC AUTONOMOUS

(Accredited by NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)
Kandlakoya, Medchal Road, Hyderabad-501 401

Department of Computer Science & Engineering (Cyber Security)

CERTIFICATE

This is to certify that the project entitled "**Virtual Security Operation Center (VSOC)"** is a Bonafide work carried out by

Y. RAGHAVA	**228R1A6264**
P. RAJEEV	**228R1A6251**
K. MODITHA	**228R1A6236**
M. AJAY	**228R1A6243**

In partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision. The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide	**Mini Project Coordinator**	**Head of the Department**
Ms. T. Usha	**Mr. Sunil Kumar Singh**	**Dr. Ashwitha Reddy**
Assistant Professor Department of CSC, CMREC, Hyderabad	Assistant Professor Department of CSC, CMREC, Hyderabad	Associate Professor & H.O.D Department of CSC CMREC, Hyderabad

EXTERNAL EXAMINER

DECLARATION

This is to certify that the work reported in the present Mini project entitled “VIRTUAL SECURITY OPERATIONS CENTRE (VSOC)” is a record of bonafide work done by us in the Department of Computer Science and Engineering, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source.We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

K. MODITHA (228R1A6236)
P. RAJEEV (228R1A6251)
Y. RAGHAVA REDDY (228R1A6264)
M. AJAY (228R1A6243)

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr.Ashwitha Reddy**, Associate Professor & HOD, Department of CSC, CMR Engineering College for their constant support.

We are extremely thankful to **Ms.T.USHA, Associate Professor**, Internal Guide, Department of CSC, for her constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with gratitude thanks to the authors of the references and other literature referred to in this Project.

We thank **Mr.Sunil Kumar Singh** , **Associate Professor** ,CSC Department ,Mini Project Coordinator for his constant support in carrying out the project activities and reviews.

We express my thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, We are very much thankful to our parents who guided us for every step.

K. MODITHA (228R1A6236)
P. RAJEEV (228R1A6251)
Y. RAGHAVA REDDY (228R1A6264)
M. AJAY (228R1A6243)

CONTENTS

TOPIC **PAGE NO**

ABSTRACT **i**

- **LIST OF FIGURES** **ii**
- **LIST OF TABLES** **iii**

1. INTRODUCTION……. **1**

1.1 Introduction of the project……..…… **1**

1.2 Purpose of the project……...…… **2**

1.3 Objectives……. **3**

1.4 Existing System with limitations..……..... **4**

1.5 Proposed system with features……. **4**

2. LITERATURE SURVEY…… **6**

3. SOFTWARE REQUIREMENTS ANALYSIS…… **8**

3.1 Problem Specification……..... **8**

3.2 Modules and Their Functionalities…….. **9**

3.3 Functional Requirements…… **11**

3.4 Non-Functional Requirements…… **13**

3.5 Feasibility Study…….. **15**

4. SYSTEM REQUIREMENTS SPECIFICATIONS……..….. **19**

4.1 System Specifications…… **19**

4.2 Software Requirements…….…. **22**

4.3 Hardware Requirements…….… **26**

5. SOFTWARE DESIGN……..... **28**

5.1 System Architecture ……..…. **28**

5.2 Data Flow Diagram……..... **30**

5.3 Control Flow Diagram…… **31**

6.UML DIAGRAMS……... 33

6.1 Class Diagram…… **33**

6.2 Use Case Diagram……..... **34**

6.3 Sequence Diagram……... **33**

6.4 Activity Diagram…… **37**

7.CODING……. 38

7.1 Alerting.py……... **38**

7.2 Dashboard.py……... **38**

7.3 Incident_response.py…… **40**

7.4 Log_management.py…… **40**

7.5 Main.py…… **41**

7.6 Threat_intelligence.py……. **43**

8.SYSTEM TESTING……..…… 44

8.1 Unit Testing……..… **44**

8.2 Integration Testing…….. **45**

8.3 Component…….. **45**

8.4 Functional……..... **45**

9.TEST CASES……. 46

10. OUTPUT……..... 48

11. CONCLUSION……...……..... 52

12. FUTURE ENHANCEMENTS……... 53

13. REFERENCES……..... 54

ABSTRACT

In the current era of increasing cyber threats, the need for real-time monitoring and rapid incident response has become paramount. A Security Operation Center (SOC) plays a critical role in safeguarding an organization's digital assets. However, traditional SOCs are resource-intensive and often inaccessible to small and medium-sized enterprises. This mini project presents the design and implementation of a Virtual Security Operation Center (VSOC) — a lightweight, scalable, and cost-effective solution for continuous threat detection and response. The vSOC integrates key security tools such as log aggregators, intrusion detection systems (IDS), and SIEM (Security Information and Event Management) frameworks within a virtualized environment. It enables centralized monitoring of network activity, log analysis, threat intelligence correlation, and incident alerting through a web-based dashboard. The project also simulates attack scenarios to demonstrate how the vSOC can detect anomalies and aid in forensic analysis. This initiative aims to provide an accessible and practical security solution for academic, research, and small organizational use, bridging the gap between cybersecurity theory and real-world application.

Keywords: Virtual Security Operation Center(vSOC), Cyber threat detection, Real- time monitoring, Incident response, Threat intelligence, Network activity monitoring, Attack simulation, Forensic analysis.

LIST OF FIGURES

FIG.NO.	DESCRIPTION	PAGENO
5.1	System Architecture	28
5.2	Data Flow Diagram	30
5.3	Control Flow Diagram	31
6.1	Class Diagram	34
6.2	Use Case Diagram	35
6.3	Sequence Diagram	36
6.4	Activity Diagram	37
10.1	Run the Code	48
10.2	Adding Logs	48
10.3	Server Started	49
10.4	Home Page	49
10.5	New Logs	50
10.6	Alerts	50
10.7	Storing logs	51
10.8	Dashboard showing all the activity.	51

LIST OF TABLES

Sno.	DESCRIPTION	PAGE NO
2.1	Literature Survey	6
4.1.	Software Requirements	24
4.2.	Hardware Requirements	26
9.1.	Test cases	46

1. INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

In the rapidly expanding digital ecosystem, cybersecurity threats have become increasingly complex and frequent. Organizations across the globe are constantly targeted by malicious actors who aim to exploit vulnerabilities for financial, political, or strategic gains. Security Operation Centers (SOCs) serve as centralized hubs that monitor, detect, investigate, and respond to security threats. However, traditional SOCs are heavily reliant on physical infrastructure and skilled personnel, which leads to high costs and limited flexibility. Moreover, the proliferation of cloud services and remote work culture demands a more scalable and responsive security model.

A Virtual Security Operation Center (VSOC) represents an evolution in cybersecurity management. It decouples the traditional hardware-based SOC components and hosts them in the cloud, enabling distributed monitoring, real-time threat intelligence, and AI-driven analytics. VSOCs provide 24x7 security monitoring with reduced dependency on on-premises assets. This project explores the architectural design, operational flow, implementation strategy, and evaluation of a VSOC using open-source technologies. The system enables integration of log sources from various platforms, correlates security events using a cloud-based SIEM, and automates responses using custom scripts and SOAR (Security Orchestration, Automation and Response) playbooks. The key benefits include cost-efficiency, scalability, remote management, and improved threat detection capabilities.

In the rapidly evolving digital ecosystem, the threat landscape is growing both in complexity and frequency. Cybercriminals are no longer limited to isolated attacks; they now utilize sophisticated tactics such as Advanced Persistent Threats (APTs), ransomware-as-a-service (RaaS), and coordinated botnet operations. These threats target critical infrastructures, intellectual property, financial systems, and personal data, creating substantial risks for both public and private sector entities.

Traditionally, organizations have relied on Security Operation Centers (SOCs) to act as the nerve center for cybersecurity. A SOC typically consists of a team of cybersecurity professionals who monitor networks and endpoints 24x7, analyze logs, detect anomalies, investigate incidents, and coordinate response efforts. However, establishing and maintaining a traditional SOC is capital-intensive. It requires significant investment in hardware (servers, storage, and dedicated monitoring systems), software (SIEM,

IDS/IPS, antivirus, firewalls), and specialized staff such as Tier 1, 2, and 3 analysts. These requirements make traditional SOCs less viable for small and medium-sized enterprises (SMEs) or institutions with constrained budgets.

With the growing adoption of cloud computing, remote work environments, and hybrid infrastructures, there is a pressing need for a more agile and scalable cybersecurity approach. This is where the concept of a Virtual Security Operation Center (vSOC) becomes highly relevant. A vSOC is essentially a cloud-native or hybrid alternative to the traditional SOC model. It virtualizes the components of a conventional SOC—log management, threat detection, incident response, and analytics—by hosting them in the cloud or in a virtualized environment.

1.2 Purpose of the Project

In the rapidly expanding digital ecosystem, organizations face a constant barrage of cybersecurity threats that are growing not only in frequency but also in sophistication. These threats include advanced persistent threats (APTs), ransomware-as-a-service (RaaS), phishing campaigns, zero-day exploits, and coordinated botnet attacks, all of which pose serious risks to financial systems, critical infrastructure, personal data, and national security. Traditionally, Security Operation Centers (SOCs) have functioned as the nerve center of cybersecurity, staffed by expert analysts and equipped with various security tools to monitor network activity, detect anomalies, and respond to incidents in real time. However, such setups are heavily reliant on dedicated physical infrastructure, costly proprietary software, and skilled cybersecurity personnel, which makes them unsustainable and impractical for small and medium-sized enterprises (SMEs), academic institutions, and resource-constrained environments.

With the increasing adoption of cloud computing, virtualization, remote work, and hybrid IT infrastructures, there is an urgent need to evolve toward a more flexible, scalable, and cost-effective security model. The concept of a Virtual Security Operation Center (VSOC) addresses this need by virtualizing traditional SOC functions and deploying them on cloud-based or hybrid platforms. A VSOC leverages open- source tools, distributed monitoring, real-time log aggregation, cloud-native Security Information and Event Management (SIEM) systems, and automated response mechanisms such as SOAR (Security Orchestration, Automation, and Response) playbooks. It enables 24/7 cybersecurity operations without requiring a centralized

physical location or a large in-house team, making it highly accessible for SMEs and educational environments.

This project aims to design and implement a functional VSOC that demonstrates the integration of key components such as log collectors, intrusion detection systems (IDS), threat intelligence feeds, and alerting mechanisms into a unified virtual system. The project also simulates real-world attack scenarios to test the system's ability to detect, analyze, and respond to threats effectively. By doing so, it bridges the gap between theoretical cybersecurity concepts and practical implementation, offering an affordable and scalable solution for continuous threat detection, centralized monitoring, forensic analysis, and rapid incident response. The primary purpose is to showcase how a VSOC can deliver comparable capabilities to traditional SOCs—without the associated cost and complexity—while being more agile, adaptable, and suitable for modern digital environments.

1.3 Objectives

The primary objective of this project is to conceptualize, design, and implement a Virtual Security Operation Center (vSOC) that serves as a scalable, cost-effective, and efficient solution for continuous cybersecurity monitoring and incident response. This initiative seeks to overcome the limitations associated with traditional, hardware-dependent Security Operation Centers (SOCs), which are often financially and operationally unfeasible for small and medium-sized organizations. The project focuses on developing a cloud-based architecture that facilitates centralized log aggregation, real-time threat detection, and automated incident response using open-source technologies. Specifically, the system will integrate components such as Security Information and Event Management (SIEM) platforms, Intrusion Detection Systems (IDS), and Security Orchestration, Automation and Response (SOAR) mechanisms. The overarching objective is to deliver a practical and adaptable security framework that bridges the gap between academic knowledge and real-world cybersecurity applications.

1.4 Existing Systems

The conventional approach to managing cybersecurity threats involves a centralized Security Operation Center(SOC) that utilizes on-premises tools and human analysts to oversee security

incidents. These SOCs often use traditional SIEM (Security Information and Event Management) systems to collect and correlate logs from different endpoints and network devices. Some organizations supplement this with manual alert handling and predefined threat rules.

The drawbacks of this system include high capital and operational expenditure, limited scalability, dependency on physical presence, and longer mean time to detect (MTTD) and mean time to respond (MTTR) to threats. In addition, traditional SOCs struggle with cloud visibility, as they are not inherently designed to monitor cloud-native applications or containers effectively.

Limitations of the Existing System

- **Scalability Constraints**: Traditional SOC infrastructures often face challenges in scaling efficiently to keep pace with the growth and complexity of enterprise networks.
- **Limited Cloud Integration**: Many existing systems provide inadequate visibility into cloud workloads and remote environments, limiting comprehensive security coverage.
- **Delayed Response:** Manual investigation workflows contribute to high dwell times, resulting in slower detection and response to security incidents.
- **Resource Intensive:** These systems require significant investment in skilled personnel, specialized hardware, and continuous maintenance efforts.
- **Siloed Data**: Disparate security tools and platforms create fragmented threat intelligence, which hampers effective event correlation and comprehensive threat detection.

1.5 Proposed System

The proposed Virtual Security Operation Center (vSOC) is a state-of-the-art, cloud-native platform designed to transform cybersecurity operations by centralizing monitoring, threat detection, incident response, and security analytics within a flexible virtual environment. Unlike traditional SOCs constrained by physical infrastructure and time-consuming manual processes, the vSOC leverages automation, artificial intelligence (AI), and machine learning (ML) to deliver scalable, real-time security intelligence and rapid incident mitigation. By seamlessly integrating essential tools such as Security Information and Event Management (SIEM), Intrusion Detection Systems (IDS), and Security Orchestration, Automation, and Response (SOAR), the system provides a unified, user-friendly dashboard that enhances visibility and operational efficiency for security teams. The platform supports automated workflows that

reduce human error and accelerate response times, while its cloud-based architecture ensures easy scalability, cost-effectiveness, and simplified maintenance. Ideal for academic research, small to medium-sized organizations, and enterprises seeking agile cybersecurity solutions, the vSOC empowers defenders to stay ahead of evolving threats in increasingly complex and distributed IT environments.

Benefits of the Proposed System

- **Real-Time Threat Detection:** Utilizes cloud-based SIEM tools to continuously collect and analyse logs from diverse sources, enabling immediate identification of potential security threats.
- **Automation:** Implements SOAR (Security Orchestration, Automation, and Response) to streamline and accelerate incident response workflows, ensuring consistent and efficient handling of security events.
- **Scalability:** Leverages cloud infrastructure platforms such as AWS and Azure to dynamically scale resources up or down based on workload demands, optimizing performance and cost.
- **Remote Accessibility:** Provides secure remote access capabilities for security analysts, allowing effective monitoring and response from any location, supporting flexible and distributed teams.

2. Literature Survey

Title	AUTHORS	JOURNALS	YEAR	ALGORITHM/APPROACH USED	CONCLUSION
A Cloud-Based Virtual SOC for Scalable Cybersecurity Operations	Ali, M. et al.	IEEE Xplore	2021	VSOC using VMs and automation	vSOC is ideal for SMBs seeking cost-saving, flexible security solutions in a connected environment.
Implementation of Virtual SOC using SIEM in Hybrid Cloud	Raj, R. & Kumar, S.	IEEE Xplore	2020	SIEM in Hybrid Cloud	Hybrid vSOC provides better control over sensitive data while still utilizing cloud benefits.
Virtual SOC: Bridging Gaps in Cybersecurity for SMEs	Smitha A	MDPI	2019	Case study on vSOC adoption	vSOC is ideal for SMEs that cannot afford full-scale physical SOCs.
Designing a Virtual Security Operations Center with SOAR	Lee, J. & Park, K.	Journal of Cybersecurity	2022	SOAR, Playbooks & Automation	Integration of SOAR improves incident response times and reduces manual workload.
Cloud-Native Security Operations Center Architecture	Patel, R.	ACM Computing Surveys	2021	Cloud SIEM and Microservices	Cloud-native SOCs improve scalability and enable real-time threat detection across distributed environments.
An Open Source Framework	Chen, L. & Zhang, Y.	IEEE Software	2020	Open Source SIEM & IDS	Open source tools enable cost-effective

for Virtual SOC Deployment					deployment of vSOCs without compromising security.
Machine Learning for Anomaly Detection in Virtual SOCs	Chen, L. & Zhang, Y.	Elsevier Journal of Information Security	2023	ML Anomaly Detection Models	ML enhances vSOC capability to detect sophisticated and novel cyber threats
Evaluation of Virtual SOC Performance in Hybrid Cloud Environments	Williams, D. & Johnson, T.	International Journal of Network Security	2022	Performance Metrics & Optimization	Hybrid vSOCs can balance security with operational efficiency in multi-cloud environments.

Table-2.1 Literature Survey

3. SOFTWARE REQUIREMENT ANALYSIS

3.1 Problem Specification

In today's digitally interconnected world, organizations face an increasing number of sophisticated and persistent cyber threats. While traditional Security Operation Centers (SOCs) offer centralized monitoring and incident response capabilities, they are often expensive, resource-intensive, and require a skilled workforce. Small and medium-sized enterprises (SMEs), academic institutions, and research environments often lack the budget and infrastructure to deploy a full-scale SOC. There is a pressing need for a scalable, cost-effective, and remotely manageable alternative that provides real-time threat detection and response without relying on dedicated physical infrastructure. This project aims to address this gap by designing and implementing a Virtual Security Operation Center (vSOC) using open-source tools.

3.1.1 Proposed System

The proposed Virtual Security Operation Center (vSOC) is a cutting-edge, cloud-native cybersecurity platform designed to modernize and streamline security operations. Unlike conventional SOCs that depend heavily on physical infrastructure and manual interventions, this vSOC leverages automation, artificial intelligence (AI), and machine learning (ML) technologies to deliver scalable, real-time threat detection and incident response.

The system integrates key security components—including Security Information and Event Management (SIEM), Intrusion Detection Systems (IDS), and Security Orchestration, Automation, and Response (SOAR)— into a unified, intuitive dashboard that enhances situational awareness and operational efficiency for security analysts. Automated workflows within the platform minimize human error and accelerate response times, while the cloud-based architecture offers flexibility, cost savings, and ease of maintenance.

Designed to serve academic research, small to medium-sized enterprises, and agile organizations, the vSOC empowers security teams to proactively defend against complex, evolving cyber threats in distributed IT environments, ensuring continuous protection and rapid mitigation.

3.2 Modules And Their Functionalities

There are six (6) Modules

- Log Aggregation Module
- Threat Detection Module
- Intrusion Detection System (IDS) Module
- Security Information and Event Management (SIEM) Module
- Security Orchestration, Automation, and Response (SOAR) Module
- Dashboard and Visualization Module

1. Log Aggregation Module

The Log Aggregation Module is responsible for collecting and centralizing logs from various data sources such as network devices, servers, applications, and endpoints. It normalizes and timestamps the incoming log data to ensure consistency and accuracy across different formats. This unified dataset provides comprehensive visibility into the entire IT infrastructure, forming the foundational data pool for analysis.

By aggregating logs in real time, the module enables continuous monitoring and prepares the data for subsequent threat detection and correlation processes. Its ability to support diverse log sources makes it adaptable to complex and distributed environments, which is critical for effective security operations.

2. Threat Detection Module

The Threat Detection Module analyzes aggregated logs and network traffic using a combination of rule-based engines and machine learning algorithms. Its main purpose is to identify suspicious patterns, anomalies, and potential threats such as intrusion attempts, malware infections, or policy violations.

This module reduces false positives by correlating multiple indicators and intelligently assessing risk. Early detection through this module empowers security teams to respond proactively to emerging cyber threats before they escalate.

3. Intrusion Detection System(IDS) Module: The IDS Module monitors network and system activity in real time to detect unauthorized or malicious behaviors. It compares observed events

against a database of known attack signatures and baseline normal activity to identify potential intrusions.

Once suspicious activity is detected, the IDS generates alerts for further investigation or automated response. This module enhances security by complementing log-based detection with real-time behavioral monitoring, thus increasing the overall threat detection coverage.

4. Security Information and Event Management (SIEM) Module

The SIEM Module acts as the centralized hub for aggregating security event data from various sources. It correlates disparate events to generate actionable alerts and contextual insights.

This module prioritizes security incidents based on severity and impact, helping security analysts to focus on the most critical threats. Its powerful event correlation and reporting capabilities improve situational awareness and enable efficient incident management.

5. Security Orchestration, Automation, and Response (SOAR) Module

The SOAR Module automates routine security operations through predefined playbooks and workflows. It coordinates incident response tasks such as alert triage, threat containment, and remediation with minimal human intervention.

Automation reduces response times and minimizes human error during incidents, allowing security teams to scale their operations efficiently. This module also integrates with other security tools to provide seamless orchestration across the security stack.

6. Dashboard and Visualization Module

The Dashboard and Visualization Module provides a centralized, user-friendly interface for security analysts to monitor the security posture in real time. It displays logs, detected threats, alerts, and system health through customizable visualizations such as charts, graphs, and heatmaps.

By consolidating relevant information in an intuitive format, this module enhances situational awareness and helps analysts make informed decisions quickly. It supports role-based access, ensuring that users see data relevant to their responsibilities. Each of these modules works together within the cloud-native vSOC platform to provide a scalable, automated, and intelligent cybersecurity solution. From data collection and threat detection to automated response and

visualization, the integrated modules empower organizations to proactively defend against evolving cyber threats with greater efficiency and accuracy.

3.3 Functional Requirements

Functional requirements describe the essential features and behaviors that the system must exhibit to meet user needs and ensure proper functioning of the Virtual Security Operation Center (vSOC). Below are the key functional requirements:

1. SIEM, IDS, and SOAR Integration
2. Centralized Web Dashboard
3. Automated Incident Response
4. Secure User Authentication and Role-Based Access
5. Cyberattack Simulation
6. Remote Access for Security Teams

1. SIEM, IDS, and SOAR Integration

The system must integrate Security Information and Event Management (SIEM), Intrusion Detection System (IDS), and Security Orchestration, Automation, and Response (SOAR) tools for comprehensive and real-time security event collection and analysis.

- Event Collection: Aggregate security events and logs from diverse sources for unified monitoring.
- Threat Analysis: Correlate and analyze events using SIEM and IDS to detect potential threats and anomalies.
- Automated Response: Utilize SOAR to automate workflows and orchestrate responses to detected incidents.
- Purpose: Provides holistic visibility into security events and accelerates threat detection and response.

2. Centralized Web Dashboard

The platform offers a centralized web dashboard to enable security teams to monitor alerts, incidents, and system health in real time.

- User Interface: Displays alerts, threat intelligence, and incident status with customizable visualizations.
- Real-Time Monitoring: Provides up-to-the-minute data for rapid situational awareness.
- Accessibility: Web-based access ensures availability across multiple devices and locations.
- Purpose: Enhances operational efficiency by consolidating security data into a single, intuitive interface.

3. Automated Incident Response

The system supports automated workflows for incident response to reduce manual intervention and accelerate mitigation.

- Predefined Playbooks: Define response procedures for common incident types.
- Orchestration: Coordinate actions across security tools and teams automatically.
- Alert Prioritization: Use AI/ML to prioritize and escalate incidents based on severity.
- Purpose: Minimizes human error and speeds up containment and remediation processes.

4. Secure User Authentication and Role-Based Access

The system must implement secure user authentication mechanisms combined with role-based access control (RBAC).

- Authentication: Require strong credentials to verify user identity before granting access.
- Role-Based Access: Restrict system capabilities and data visibility based on user roles and responsibilities.
- Audit Trails: Log all user actions for accountability and forensic analysis.
- Purpose: Ensures that only authorized personnel access sensitive data and perform critical operations.

5. Cyberattack Simulation

The platform must allow simulated cyberattack exercises to test system defenses and team readiness.

- Simulation Tools: Generate realistic attack scenarios reflecting evolving threat landscapes.
- Testing: Validate detection capabilities, response effectiveness, and system resilience.
- Reporting: Provide detailed results and recommendations for improvement.

- Purpose: Enables proactive identification of gaps and continuous improvement of security posture.

6. Remote Access for Security Teams

The system must provide secure remote access to enable security teams to operate and respond from distributed locations.

- Secure Connectivity: Use encrypted channels and multi-factor authentication for remote logins.
- Full Functionality: Remote users should have access to the same monitoring and response tools as onsite teams.
- Availability: Ensure reliable uptime and performance for remote access services.
- Purpose: Supports flexible operations and timely incident management regardless of team location.

These functional requirements define the core capabilities necessary for the vSOC to deliver real-time, scalable, and automated cybersecurity operations. By integrating advanced tools, centralized monitoring, secure access controls, simulation capabilities, and remote connectivity, the system empowers security teams to effectively detect, analyze, and respond to evolving cyber threats.

3.4 Non-Functional Requirements

Non-functional requirements define the system's operational qualities, ensuring that the Virtual Security Operation Center (vSOC) performs well under various conditions. These requirements focus on how the system behaves rather than what it does.

1. Scalability
2. Availability
3. Security
4. Usability
5. Flexibility
6. Performance

1. Scalability

The vSOC must be capable of handling increasing volumes of security data and growing numbers of users without any degradation in performance.

- Elastic Resource Management: Dynamically allocate computational and storage resources based on demand.
- Cloud-Native Architecture: Leverage cloud services to scale horizontally and vertically as needed.
- Purpose: Ensures consistent, high-performance operation even as monitoring scope and user base expand

2. Availability

High availability and fault tolerance are critical to maintain continuous security monitoring and incident response.

- Redundancy: Deploy redundant components and failover mechanisms to prevent single points of failure.
- Disaster Recovery: Implement backup and recovery plans to minimize downtime during outages.
- Purpose: Guarantees uninterrupted operation and rapid recovery in the event of hardware or software failures

3. Security

The system must safeguard sensitive security data in transit and at rest, protecting against unauthorized access and data breaches.

- Data Encryption: Use strong encryption protocols (e.g., TLS for communication, AES for storage) to protect data confidentiality.
- Access Controls: Enforce secure authentication mechanisms and role-based access control to restrict data and system access.
- Purpose: Protects sensitive information from interception, tampering, and unauthorized use.

4. Usability

The vSOC should provide a responsive, user-friendly interface compatible with various devices to facilitate efficient security operations.

- Cross-Device Compatibility: Support desktop browsers, tablets, and smartphones for monitoring and response activities.
- Intuitive Design: Offer clear visualization, easy navigation, and customizable dashboards tailored to user roles.

- Purpose: Enhances user productivity and satisfaction by simplifying complex security data interactions.

5. Flexibility

Deployment options should accommodate virtualized and cloud environments to optimize resource usage and reduce costs.

- Deployment Models: Support on-premises virtual machines, private/public cloud infrastructure, and hybrid setups.
- Integration Capability: Easily integrate with existing IT infrastructure and security tools.
- Purpose: Enables organizations to adapt deployment based on budget, compliance, and operational preferences.

6. Performance

The system must deliver fast, real-time processing of security events to enable timely detection and response.

- Low Latency Processing: Minimize delays in event ingestion, analysis, and alert generation.
- Efficient Resource Utilization: Optimize compute and network resources for continuous operation under high load.
- Purpose: Ensures rapid incident identification and mitigation without bottlenecks or system lag.

By implementing these non-functional requirements, the vSOC ensures scalable, resilient, secure, and user- friendly cybersecurity operations suitable for modern and distributed IT environments.

3.5 Feasibility Report

The Feasibility Report is a detailed analysis that evaluates the viability of the proposed Virtual Security Operation Center (vSOC) project across several critical dimensions. It assesses technical feasibility by determining if the required technology and resources are sufficient and available. Economic feasibility is analyzed to ensure the project is financially practical, weighing costs, benefits, and long-term savings. Operational feasibility focuses on whether the system can be smoothly integrated and sustained in practice. Legal feasibility ensures that the platform complies with relevant regulations, and schedule feasibility evaluates the project timeline for

realistic completion. This report helps stakeholders make informed decisions by identifying potential risks, benefits, and resource allocations.

Projects are initiated for two broad reasons: to solve a problem or to capitalize on an opportunity. Five key considerations involved in the vSOC feasibility analysis are:

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility
- Legal Feasibility
- Schedule Feasibility

1. Technical Feasibility

The vSOC is technically feasible due to the availability of mature tools and skilled expertise in the organization.

- Cloud Infrastructure: Cloud-native platforms like AWS and Azure offer scalable, reliable environments for deploying SIEM, IDS, and SOAR components.
- Open-Source Tools: Mature open-source solutions (e.g., Wazuh, Zeek, TheHive) reduce development time and allow for easy integration.
- Team Expertise: The development team has demonstrated proficiency in network security, cloud orchestration, and virtualization technologies, supporting seamless implementation.

Conclusion: The project leverages proven tools and skilled personnel, making it technically feasible for secure and scalable deployment.

2. Economic Feasibility

The economic feasibility demonstrates that the vSOC is a cost-effective solution, particularly for institutions with limited budgets.

- Reduced Capital Expenditure: Utilizing cloud infrastructure avoids large upfront investments in physical hardware.
- Open-Source Licensing: Relying on community-supported tools eliminates licensing fees.
- Lower Maintenance Costs: Automation of monitoring and incident response lowers ongoing operational expenses.

Conclusion: The vSOC is economically viable due to its efficient use of resources, making it suitable for cost- sensitive environments such as academic institutions and SMEs.

3. Operational Feasibility

Operational feasibility evaluates how well the system can be managed post-deployment, considering ease of use and adaptability.

- User-Friendly Interface: The vSOC includes a centralized, intuitive dashboard that simplifies threat monitoring and response.
- Automated Workflows: Automated alerts and playbooks minimize human effort and improve response time.
- Remote Accessibility: Secure remote access supports hybrid or distributed security teams, enhancing flexibility.

Conclusion: With a focus on automation and usability, the vSOC is operationally feasible and can be effectively adopted by diverse security teams.

4. Legal Feasibility

Legal feasibility ensures the system adheres to applicable laws and standards for cybersecurity and data privacy.

- Regulatory Compliance: The vSOC is designed to comply with data protection regulations like the GDPR, India's IT Act, and other cybersecurity policies.
- Secure Data Handling: Data encryption, access controls, and secure audit logging are implemented to support legal defensibility.
- Governance: Access and activity logs are maintained for auditing and compliance reporting.

Conclusion: Legal feasibility is achieved through the platform’s adherence to cybersecurity and privacy regulations, ensuring lawful data handling practices.

5. Schedule Feasibility

Schedule feasibility assesses whether the project can be completed within the desired timeframe using phased development.

- Modular Development: The vSOC can be built in stages (e.g., SIEM first, then SOAR), allowing flexibility in scheduling.
- Cloud Acceleration: Cloud infrastructure eliminates provisioning delays, enabling rapid deployment and iteration.
- Project Management: With proper resource planning and milestone tracking, delivery goals are attainable.

4. SYSTEM REQUIREMENTS SPECIFICATIONS

4.1 System Specifications

The Virtual Security Operation Center (vSOC) is designed to provide scalable, cost-effective, and secure threat monitoring and incident response by leveraging cloud-native infrastructure and open-source tools. The system architecture is composed of hardware, software, and supporting environments required for efficient deployment and operation. This section outlines the detailed system requirements, categorized into five major components:

- Endpoint Hardware Environment
- Base Software Stack
- Development Tools and Runtime Environment
- Optional and Supporting Software
- Visualization and Reporting Tools

1. Endpoint Hardware Environment

The endpoint systems serve as the primary environment for deploying, running, and testing the vSOC modules. The hardware specifications are optimized for lightweight operation and cross-platform compatibility.

Key Functionalities:

Processor (CPU):

1. Minimum: Dual-Core 2.0 GHz or equivalent
2. Supports smooth operation of scanning tools and log processing scripts

Memory (RAM):

1. 4 GB
2. Sufficient for running lightweight IDS/SIEM modules and log collectors

Storage:

1. Minimum of 1 GB of free disk space
2. Used for storing logs, reports, and dependencies

Display:

1. 1024×768 resolution
2. Ensures compatibility with visualization dashboards

Network Connectivity:

1. Basic internet connection
2. Required for downloading packages and remote updates

Conclusion: The system is designed to operate on modest hardware, making it accessible for academic and SME- level deployment environments.

2. Base Software Stack

The core software components required to run the vSOC system include operating systems, programming languages, and fundamental development tools.

Key Functionalities:

Operating System Support:

- Windows 10/11
- Linux (Ubuntu/Debian)
- macOS

Offers flexibility for developers and administrators

Programming Language:

1. Python 3.8 or higher
2. Chosen for its versatility in scripting, automation, and integration

Python Libraries:

1. configparser (standard library)
2. Additional modules as required per implementation (e.g., requests, pandas, socket, etc.)

Command Line Tools:

1. Terminal (Linux/macOS) or Command Prompt / PowerShell (Windows)
2. Required for script execution, testing, and debugging

Package Manager:

1. pip for Python dependency management

Conclusion: The base software stack ensures cross-platform support, efficient scripting, and ease of integration with open-source threat detection tools.

3. Development Tools and Runtime Environment

This environment supports the coding, testing, and debugging of core security modules. Key Functionalities:

Development Environments (IDEs):

1. Visual Studio Code
2. PyCharm
3. Sublime Text

Provide robust support for Python development and debugging

Version Control:

1. Git
2. Essential for collaborative development and project version tracking

Web Server Software (for simulation):

1. Apache or Nginx
2. Used to simulate real-world web application environments for penetration testing and monitoring

Conclusion: The development and runtime setup ensures rapid development and testing while supporting real- world scenario simulations.

4. Optional and Supporting Software

These tools extend the vSOC functionality and enable integration with existing infrastructure or enhanced analytics.

Key Functionalities:

ELK Stack (Elasticsearch, Logstash, Kibana):

1. Centralized log collection, parsing, and visualization
2. Enhances monitoring and data-driven analysis

Splunk: Commercial alternative for log ingestion and threat visualization

Database Management Systems (DBMS):

1. MySQL or PostgreSQL
2. Required for storing structured data such as scan logs or user activity metrics

Java Runtime:

1. Required for select security tools and integration platforms
2. Ensures compatibility with Java-based modules

Conclusion: Optional tools provide extensibility and are useful for organizations needing deeper analytics, database support, or advanced log management.

5. Visualization and Reporting Tools

These tools help present data in understandable, actionable formats for decision-makers and analysts. Key Functionalities:

Graphviz:

- Visual representation of scan paths, attack graphs, and relationships between events

Matplotlib:

- Used for generating custom plots and statistical summaries from log data

Conclusion: These tools enhance the readability and interpretation of security data, supporting effective decision- making and threat prioritization.

The vSOC system specification ensures a lightweight yet robust platform suitable for modern threat detection and response. The modular design supports broad compatibility with hardware and software environments, while the optional tools provide flexibility for advanced use cases. By combining minimal hardware requirements with powerful open-source software, the vSOC architecture is scalable, sustainable, and ideal for both academic and enterprise security operations.

4.2 Software Requirements

Operating Systems

- Windows 10/11: Compatible for development and execution of the vSOC environment.
- Linux (Ubuntu/Debian): Preferred for open-source compatibility and server-side execution.
- MacOS: Supported for development and lightweight monitoring tools.

Programming Languages

Python:

- Primary language for scripting, automation, and core module development (Python 3.8 or higher).

Java:

- Required for compatibility with select security tools and frameworks.

Frameworks & Libraries

configparser:

- Standard Python library used for reading configuration files.

matplotlib:

- Used for generating visual plots and scan result summaries.

Graphviz:

- Useful for building attack graphs and network flow diagrams.

Additional Python Libraries (as needed):

- Depending on the implementation (e.g., requests, pandas, flask, etc.).

Tools

Visual Studio Code / PyCharm / Sublime Text:

- IDEs for Python development, debugging, and testing.

Terminal / Command Prompt / PowerShell:

- For executing scripts and system commands across OS platforms.

pip (Python Package Installer):

- Used to manage Python library dependencies.

Git:

- Version control tool for collaborative development and code tracking.

ELK Stack (Elasticsearch, Logstash, Kibana):

- For centralized logging, log parsing, and dashboard visualization.

Splunk:

- Optional SIEM tool for enhanced threat monitoring and analysis.

Supporting Software

Web Server Software:

Apache or Nginx – To host web applications and simulate real-world server environments.

Category	Software/Tool	Description
Programming Languages	Python	Python is the primary language used for developing the VSOC application, known for its simplicity and extensive libraries.
Framework & Libraries	Asyncio	A Python library used for writing concurrent code using the async/await syntax, enabling real-time log monitoring.
	Logging	Python's built-in logging module for structured logging and log management, essential for tracking application events.
	Watchtower	A logging handler that sends logs to AWS CloudWatch, facilitating centralized log management (if cloud integration is desired).
Data Analysis	Pandas	A powerful data manipulation and analysis library for Python, useful for processing log data and threat analysis.
	NumPy	A library for numerical computations in Python, often used alongside Pandas for data analysis tasks.
Incident Response	Custom Modules	Custom Python modules for incident response actions, automating responses to detected threats.

Threat Intelligence	Custom Modules	Modules for updating and managing threat intelligence data, ensuring the application is aware of emerging threats.
Development Tools	Visual Studio Code	A popular code editor that supports Python development with extensions for linting, debugging, and version control.
	PyCharm	An integrated development environment (IDE) specifically for Python, providing advanced features for code management and debugging.
Version Control	Git	A version control system for tracking changes in the codebase, facilitating collaboration and code management.
Deployment	Docker	A platform for developing, shipping, and running applications in containers, ensuring consistency across environments.
	Flask	A lightweight web framework for Python, used to create the dashboard and API for the VSOC application.
Monitoring	Local File System	Utilizes local file storage for logging, allowing for easy access and review of historical logs.

Table-4.1 Software Requirements

4.3 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Dual-Core 2.0 GHz or equivalent	Quad-Core or higher
RAM	4 GB	8 GB or higher
Display	1024×768 resolution	Full HD (1920×1080)
Network	Basic internet connection	Stable broadband for real-time updates and package downloads
Storage	1 GB free disk space	SSD with 20 GB free space

Table-4.2: Hardware Requirements

End users need a standard computing environment capable of supporting Python-based applications. A stable internet connection is required for downloading packages, updates, and interacting with optional remote services such as centralized logging or cloud platforms.

Component	Minimum Requirement	Recommended Requirement
Ethereum Node Provider	8 GB RAM	Dedicated Ethereum full node via services like Infura or Alchemy
Server (Backend & Verification)	Cloud-Based (AWS, Azure, or GCP)	High-performance VPS with 16 GB RAM and SSD storage
Processor	Quad-Core	8-Core (for high-volume transaction handling)
Storage	100 GB (for logs and blockchain records)	500 GB SSD (for performance and scalability)

Table-4: Smart Contract Deployment & Blockchain Infrastructure

If the system is later extended for blockchain integration (e.g., for notarization, logging, or certificate anchoring), it is recommended to use a robust backend and Ethereum infrastructure to ensure optimal performance and reliability.

5. Software Design

5.1 System Architecture

The Virtual Security Operations Center (vSOC) is a modular system designed to simulate and analyze cybersecurity events in a controlled environment. Its architecture is composed of several interdependent components, each playing a crucial role in the workflow. The Configuration Manager is responsible for loading key parameters—such as the number of events to simulate—from a configuration file (config.ini). The Rule Engine serves as the core detection module, loading predefined detection rules that help identify suspicious patterns within simulated data. The Event Simulator generates synthetic cybersecurity events, mimicking real- world attack scenarios, and applies the loaded rules to these events in order to detect potential threats. Once a threat is identified, the Incident Manager records the incident details and compiles them into a comprehensive final report. This modular design ensures flexibility, ease of testing, and extensibility for developing and validating cybersecurity monitoring techniques.

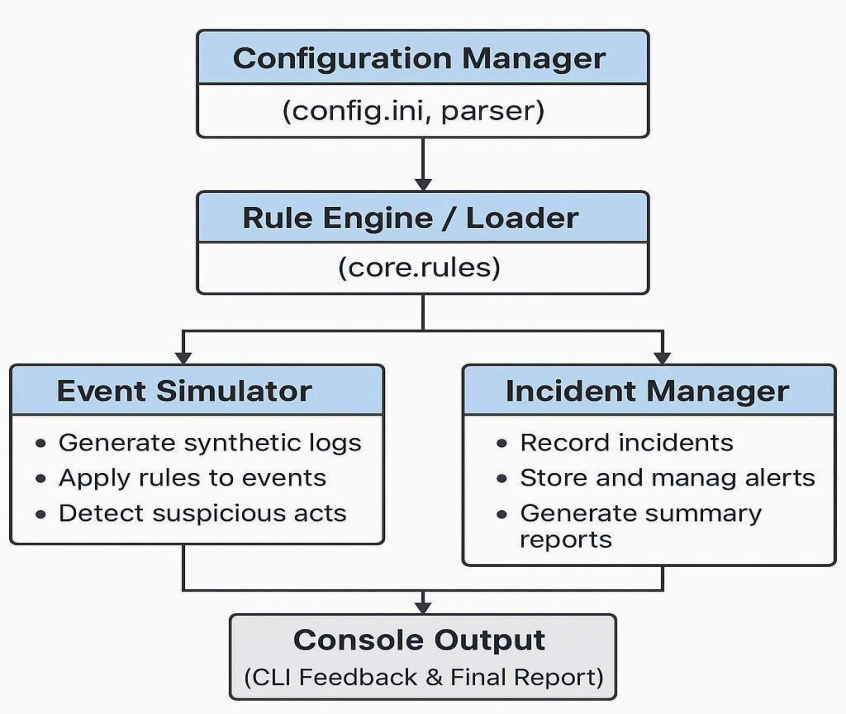


Figure-5.1 System Architecture

- **Configuration Manager:** Loads parameters like the number of events from config.ini.

- **Rule Engine:** Loads detection rules to identify suspicious patterns in events.
- **Event Simulator:** Generates synthetic events and applies rules to detect threats.
- **Incident Manager**: Records detected incidents and generates a final report.

Key Features of the System

- **Modular Architecture**: The system is divided into independent, reusable components, allowing for flexibility, scalability, and easier maintenance.
- **Event Simulation**: Generates synthetic cybersecurity events to simulate real-world attack scenarios for testing and training purposes.
- **Configurable Parameters**: Uses a configuration file (config.ini) to set simulation parameters such as the number of events, enhancing customization.
- **Rule-Based Detection Engine**: Integrates a rule engine that loads and applies customizable detection rules to identify suspicious patterns in event data.
- **Automated Threat Detection**: Automatically flags events that match threat criteria, helping simulate real-time security monitoring.
- **Incident Management and Reporting**: Detected incidents are logged, categorized, and compiled into a structured final report for analysis and documentation.
- **Extensibility**: Designed to easily integrate additional modules or detection techniques, making it suitable for research and development.

5.2 Data Flow Diagram

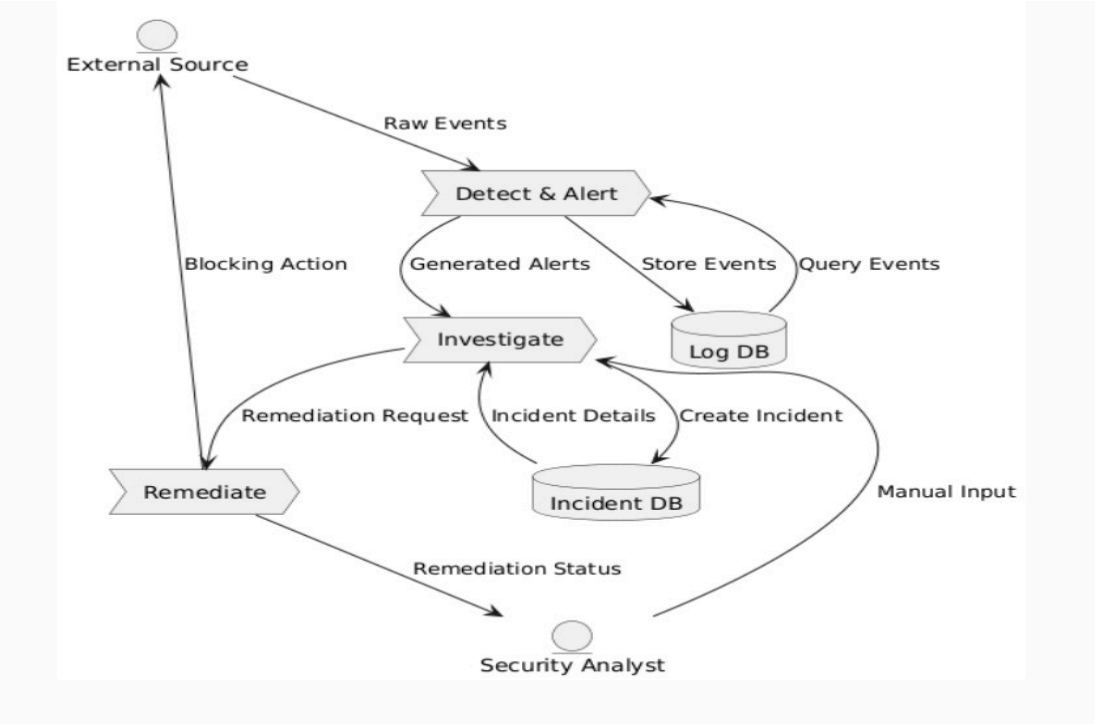


Figure-5.2 Data Flow Diagram

A Data Flow Diagram (DFD) is a visual representation of how data moves through a system. It shows how inputs (such as user data) are processed, stored, and output within the system. The DFD for the Virtual Security Operation Center (VSOC) illustrates the interaction between various cybersecurity entities, including the Network, Cloud, Endpoint Detection and Response (EDR), Security Information and Event Management (SIEM), Threat Intelligence, and Security Analysts. The Network and Cloud act as data sources, feeding information into the central VSOC. The VSOC processes and analyzes this data using integrated tools like EDR, SIEM, and Threat Intelligence platforms. Finally, the output—such as alerts, reports, or incident responses—is directed to Security Analysts, who act upon the intelligence to protect and monitor the digital environment effectively.

5.3 Control Flow Diagram

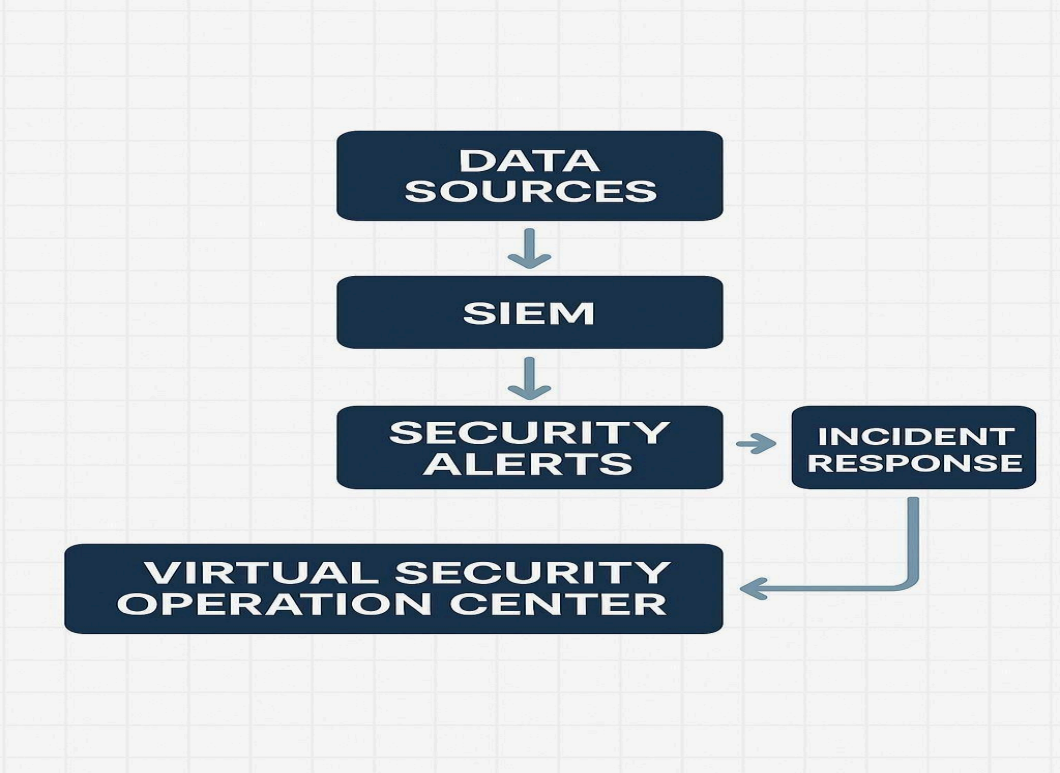


Figure-5.3 Control Flow Diagram

A Control Flow Diagram (CFD) represents the logical flow of control within a system, showcasing the decision- making process at each stage. The CFD for the Virtual Security Operation Center (VSOC) illustrates how data from multiple sources is ingested, analyzed for threats, and handled through alerting and response mechanisms, all coordinated by a centralized virtual team.

Data Sources

- Multiple data feeds such as logs, network traffic, endpoint telemetry, and cloud services send raw security data.
- These sources can include firewalls, intrusion detection systems (IDS), antivirus software, and cloud service APIs.

SIEM (Security Information and Event Management)

- The raw data is aggregated and normalized by the SIEM platform.

- Correlation rules and analytics are applied to identify suspicious behaviors or anomalies.
- If a pattern matches a known threat signature or anomalous activity, the SIEM generates an alert.

Security Alerts

- The alerts generated by SIEM are queued and prioritized based on severity and risk indicators.
- Basic triaging is performed to determine whether the alert is a false positive or needs escalation.

Incident Response

- If an alert is deemed valid and serious, it triggers the Incident Response (IR) process.
- IR teams investigate the context, determine the scope, and initiate containment, eradication, and recovery actions.
- The findings are documented, and mitigation steps are communicated to stakeholders.

Virtual Security Operation Center (VSOC)

- The VSOC acts as the centralized command center where analysts, tools, and processes are coordinated virtually.
- Analysts validate alerts, oversee response actions, and update detection rules to enhance future threat handling.
- Feedback from incident handling is fed back into the SIEM rules and data correlation logic to improve detection fidelity.
- The VSOC also manages communication, reporting, and compliance tracking remotely.

Feedback Loop & Continuous Improvement

- The outcomes of incident investigations feed back into the alerting and detection pipeline to refine correlation rules.
- Lessons learned are used to strengthen security posture and reduce future response times.

6. UML Diagrams

Unified Modelling Language (UML) diagrams are visual representations used in software development to design, analyse, and document systems. They provide a structured view of system components, processes, and interactions. UML diagrams are categorized into structural diagrams (e.g., Class Diagram, Component Diagram) that define system architecture, and behavioural diagrams (e.g., Use Case Diagram, Sequence Diagram, Activity Diagram) that illustrate system functionality. They help in visualizing system workflows, improving communication among stakeholders, and ensuring clarity in software design. UML is widely used in object-oriented programming to enhance software quality, maintainability, and scalability, making it a crucial tool in system development.

6.1 Class Diagram

The UML diagram represents a comprehensive flow of a cybersecurity incident management system, detailing how different components collaborate to detect, report, and respond to security threats. The process starts with a User who accesses the system by logging in through the login() function. An Admin—typically responsible for managing security events—uses the Web Portal to submit security-related issues or monitoring requests using the submitRequest() method. This action sends a request through the API Gateway, which acts as a secure intermediary, forwarding the request to internal systems using the forwardSecurityRequest() function.

The forwarded request reaches the Incident Response System, where it is processed and logged using the logIncident() method. This logging is crucial for documentation, auditing, and initiating further response activities. The Incident Response System then interacts with the Threat Detection Engine, which takes charge of the actual threat analysis. It performs three main operations: raiseAlert() to flag potential issues, fetchLogData() to retrieve relevant system or activity logs, and analyzeLogs() to examine those logs for suspicious patterns or behaviors. If a threat is confirmed or suspected, the system communicates with the Notification Service, which executes the sendAlert() function to broadcast warnings or alerts to administrators, security teams, or integrated systems.

This diagram highlights a robust and layered security process where detection, analysis, logging, and communication are tightly integrated. The modular design allows for scalability, as new services (like machine learning-based anomaly detection) can be introduced at the detection or response layers. Overall, the system ensures proactive and reactive capabilities in handling cyber threats effectively and efficiently.

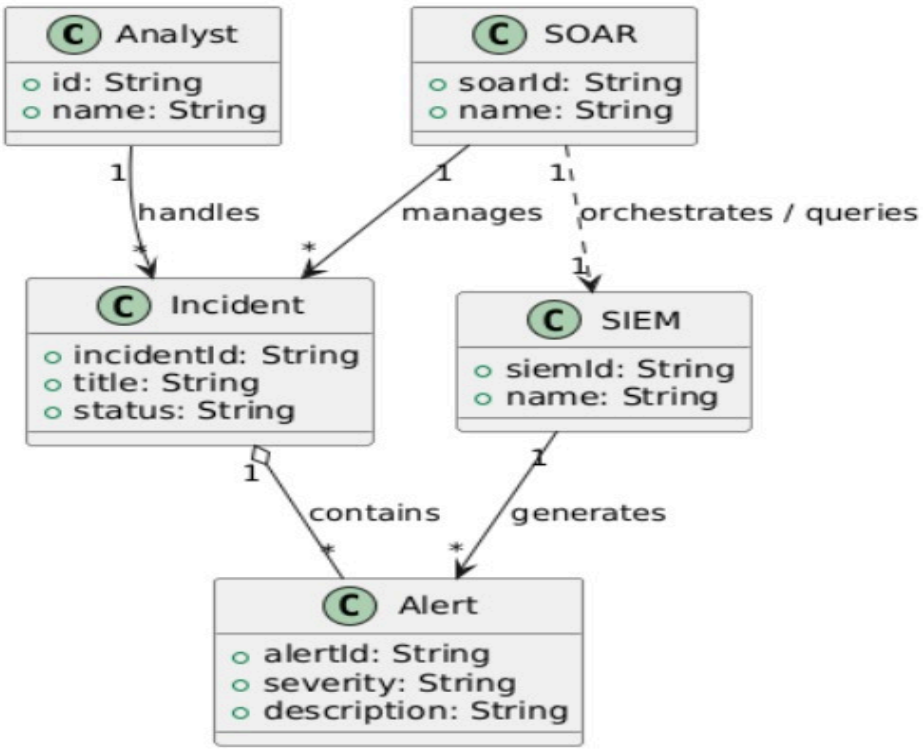


Figure-6.1 Class Diagram

6.2 Use Case Diagram

The UML diagram formally represents a cybersecurity incident management system, outlining both the system's internal components and the roles of external actors. The process is initiated by a User, who gains access to the system through the login() function. Once authenticated, the user interacts with the Web Portal, which serves as the primary interface for system operations. Administrative tasks, such as submitting a security request, are specifically handled by the Admin role through the submitRequest() operation. This request is processed by the API Gateway, which uses the forwardSecurityRequest() function to route the information to internal subsystems.

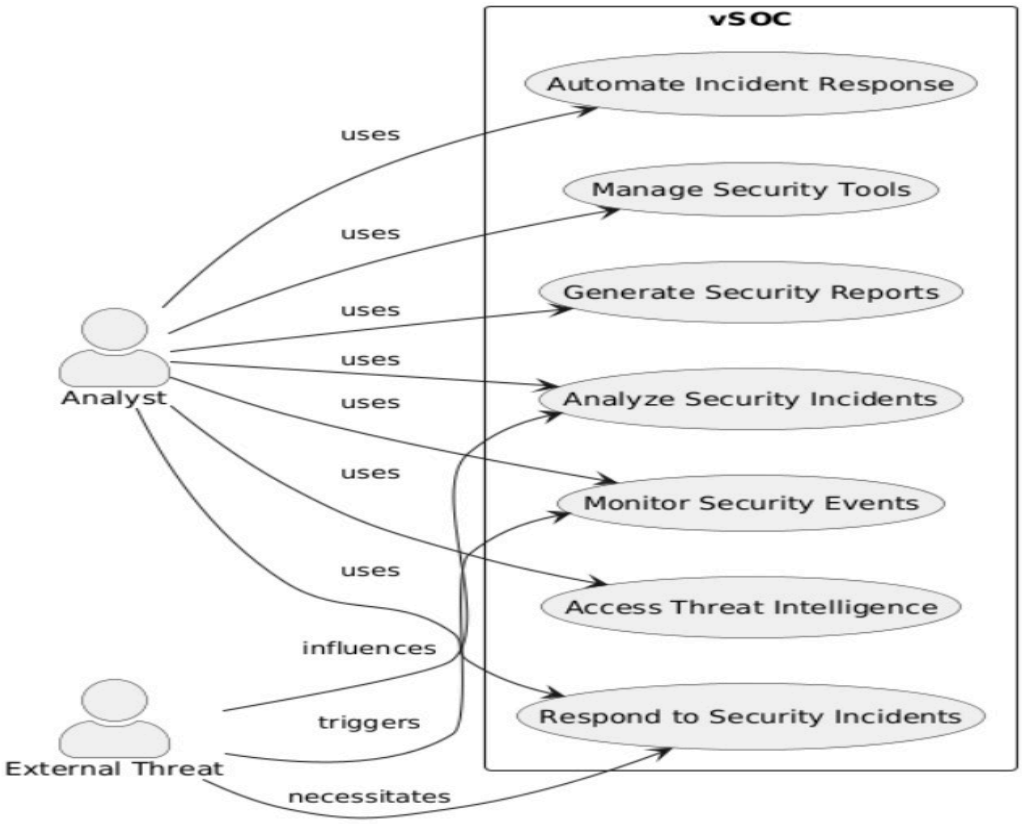


Figure-6.2 UseCase Diagram

Upon receiving the request, the Incident Response System is responsible for managing and recording the incident using the logIncident() method. It subsequently coordinates with two critical components. The Threat Detection Engine is engaged to carry out threat analysis operations, including raiseAlert(), fetchLogData(), and analyzeLogs(), which collectively enable the identification, retrieval, and examination of potential threat indicators. Concurrently, the Notification Service is activated to deliver alerts via the sendAlert() function, ensuring timely communication of threat-related information to relevant stakeholders.

This diagram effectively delineates the responsibilities of both user roles and system components, illustrating a clear and organized workflow for handling security incidents. The modular design promotes operational efficiency, security, and scalability within the incident response framework.

6.3 Sequence Diagram

A Sequence Diagram depicts the step-by-step flow of interactions between different system components in a time-sequenced manner. In the notarization system, the sequence of operations ensures the secure and verifiable signing process.

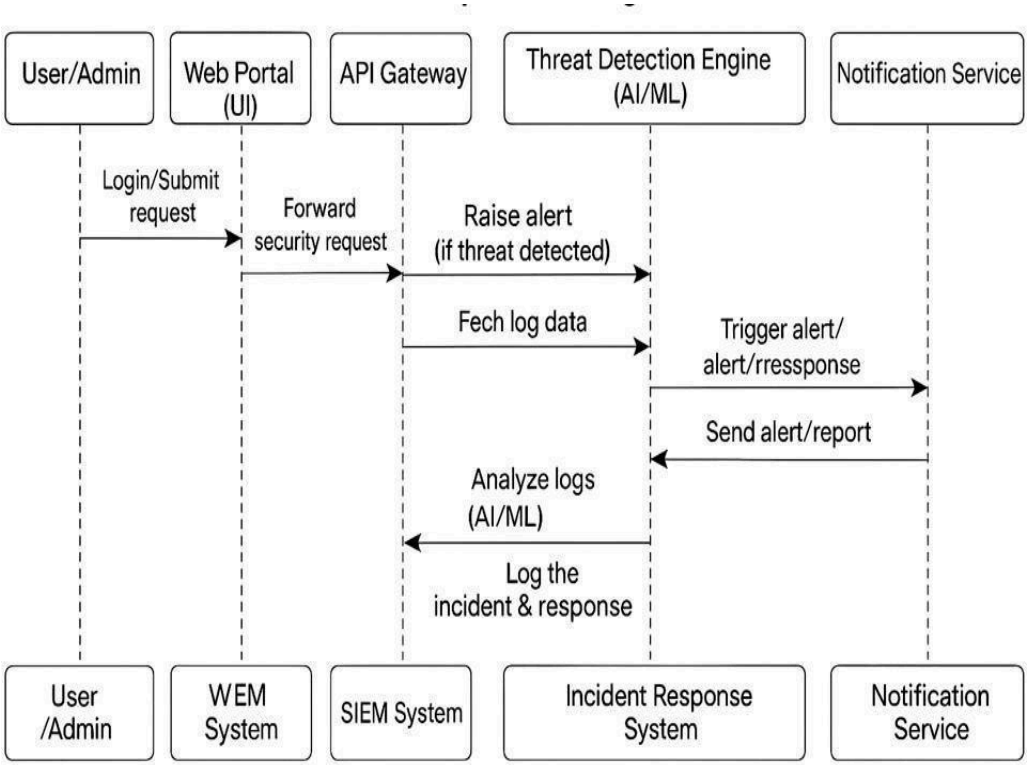


Figure-6.3 Sequence Diagram

The UML sequence diagram illustrates the flow of a security threat detection and response process in an enterprise environment. The process begins when a User/Admin initiates a login or submits a request through the Web Portal (UI). This request is forwarded as a security request to the API Gateway, which acts as an intermediary to manage and route requests. The API Gateway then communicates with the Threat Detection Engine (AI/ML), which analyzes the request using artificial intelligence and machine learning to detect any potential threats.

If a threat is identified, the Threat Detection Engine raises an alert and proceeds to fetch log data. This log data is sent to the Incident Response System, where it is further analyzed using AI/ML tools to determine the nature and scope of the incident. The response system logs both the

incident and the corresponding response for future auditing and reference. Following this, an alert or report is sent to the Notification Service, which in turn triggers notifications or alerts to the appropriate personnel or systems. This sequence ensures real-time threat detection, analysis, and notification, helping to secure the system proactively.

The actors involved include the User/Admin, Web Portal, API Gateway, Threat Detection Engine, Incident Response System, SIEM System, and Notification Service, each playing a vital role in maintaining the system's security posture through automated and intelligent workflows.

6.4 Activity Diagram

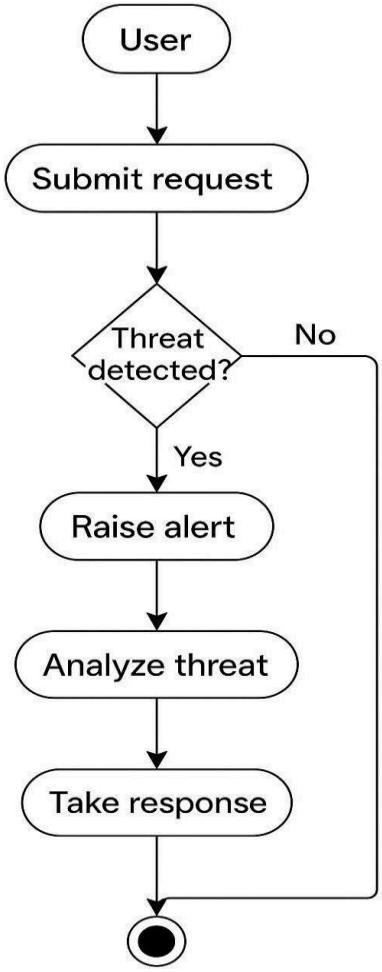


Figure-6.4 Activity Diagram

7. Coding

7.1 Alerting.py

```
def send_alert(threat):
    try:
        # Simple alert via print, but replace with email/SMS/Slack
integration here
        print(f"ALERT: Threat detected - {threat}")
    except Exception as e:
        print(f"Error sending alert: {e}")
```

7.2 Dashboard.py

```
from flask import Flask, jsonify, render_template_string
import threading
import asyncio
import main

app = Flask(__name__)

# Run VSOC monitor in separate thread using asyncio
def run_monitor():
    asyncio.run(main.main())

monitor_thread = threading.Thread(target=run_monitor, daemon=True)
monitor_thread.start()

# Updated dashboard HTML template (inline CSS/JS)
dashboard_html = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>VSOC Dashboard</title>
    <style>
        body { font-family: Arial, sans-serif; background: #1a1a2e; color:
#eaeaea; margin: 0; padding: 20px; }
        h1 { color: #0af; }
        .section { margin-bottom: 20px; }
        .logs, .threats, .alerts { background: #16213e; padding: 10px;
border-radius: 5px; max-height: 150px; overflow-y: auto; }
        .log-item { border-bottom: 1px solid #0af; padding: 5px; }
```

```
        .no-data { font-style: italic; color: #ccc; }
    </style>
</head>
<body>
    <h1>Virtual Security Operations Center (VSOC) Dashboard</h1>

    <div class="section">
        <h2>Logs</h2>
        <div id="logs" class="logs"></div>
    </div>

    <div class="section">
        <h2>Threats Detected</h2>
        <div id="threats" class="threats"></div>
    </div>

    <div class="section">
        <h2>Alerts</h2>
        <div id="alerts" class="alerts"></div>
    </div>

    <script>
        async function fetchData() {
            try {
                const response = await fetch('/api/status');
                const data = await response.json();

                // Populate logs
                const logsDiv = document.getElementById('logs');
                logsDiv.innerHTML = data.logs.length ? data.logs.map(log
=> `<div class="log-item">${log}</div>`).join('') : '<div
class="no-data">No logs available</div>';

                // Populate threats
                const threatsDiv = document.getElementById('threats');
                threatsDiv.innerHTML = data.threats.length ?
data.threats.map(t => `<div class="log-item">${t}</div>`).join('') : '<div
class="no-data">No threats detected</div>';

                // Populate alerts
                const alertsDiv = document.getElementById('alerts');
                alertsDiv.innerHTML = data.alerts.length ?
data.alerts.map(a => `<div class="log-item">${a}</div>`).join('') : '<div
class="no-data">No alerts</div>';
            } catch (e) {
                console.error('Error fetching dashboard data:', e);
            }
```

```
        }

        // Refresh every 3 seconds
        setInterval(fetchData, 3000);
        fetchData();
    </script>
</body>
</html>
"""

@app.route('/')
def dashboard():
    return render_template_string(dashboard_html)

@app.route('/api/status')
def api_status():
    # Serve shared data for frontend dashboard
    return jsonify(main.shared_state)

if __name__ == "__main__":
    # Run Flask app — now the VSOC monitoring runs in a thread alongside
    app.run(debug=True, use_reloader=False)
```

7.3 Incident_Response.py

```
def respond_to_threat(threat):
    try:
        # Simulate incident response actions like isolating endpoint,
blocking IP etc.
        print(f"Responding to threat: {threat}")
    except Exception as e:
        print(f"Error during incident response: {e}")
```

7.4 Log_Management.py

```
import asyncio

async def collect_logs():
    try:
        # Simulate asynchronous log collection from multiple sources
        await asyncio.sleep(1)  # Simulate delay
        logs = [
            "User login succeeded",
```

```
            "File accessed",
            "threat detected: suspicious login attempt",
            "System scan completed",
            "threat detected: unauthorized access attempt",
            "User logout succeeded"
        ]
        return logs
    except Exception as e:
        print(f"Error collecting logs: {e}")
        return []

def analyze_logs(logs):
    try:
        threats = []
        for log in logs:
            # Simple detection: if 'threat' is in log string, flag it
            if "threat" in log.lower():
                threats.append(log)
        return threats
    except Exception as e:
        print(f"Error analyzing logs: {e}")
        return []
```

7.5 Main.py

```
import asyncio
import logging
from logging.handlers import RotatingFileHandler
import log_management
import threat_intelligence
import incident_response
import alerting

# Shared state for dashboard or API exposure
shared_state = {
    "logs": [],
    "threats": [],
    "alerts": []
}

def setup_logging():
    logger = logging.getLogger("VSOC")
    logger.setLevel(logging.INFO)

    # Console handler with a simple format
```

```
    ch = logging.StreamHandler()
    ch.setLevel(logging.INFO)
    ch_formatter = logging.Formatter('%(asctime)s - %(levelname)s -
%(message)s')
    ch.setFormatter(ch_formatter)

    # Rotating file handler (5 MB max, keep 3 backups)
    fh = RotatingFileHandler("vsoc.log", maxBytes=5*1024*1024,
backupCount=3)
    fh.setLevel(logging.INFO)
    fh_formatter = logging.Formatter('%(asctime)s - %(levelname)s -
%(message)s')
    fh.setFormatter(fh_formatter)

    # Clear old handlers and add new ones
    if logger.hasHandlers():
        logger.handlers.clear()
    logger.addHandler(ch)
    logger.addHandler(fh)

    return logger

async def monitor_loop(logger):
    try:
        logger.info("VSOC Monitoring System started.")

        while True:
            # Collect logs asynchronously
            logs = await log_management.collect_logs()

            # Update shared logs state
            shared_state["logs"] = logs
            for log in logs:
                logger.info(f"Log collected: {log}")
            threats = log_management.analyze_logs(logs)
            shared_state["threats"] = threats
            if threats:
                for threat in threats:
                    logger.warning(f"Threat detected: {threat}")
            else:
                logger.info("No threats detected.")
            alerts = []
            for threat in threats:
                try:
                    incident_response.respond_to_threat(threat)
                    alerting.send_alert(threat)
                    alert_msg = f"Threat responded to: {threat}"
```

```
                    alerts.append(alert_msg)
                    logger.info(alert_msg)
                except Exception as e:
                    logger.error(f"Error responding to threat '{threat}':
{e}")

            shared_state["alerts"] = alerts

            # Update threat intelligence periodically
            try:
                threat_intelligence.update_threat_intelligence()
                logger.info("Threat intelligence updated.")
            except Exception as e:
                logger.error(f"Error updating threat intelligence: {e}")

            # Wait before next iteration (e.g., 10 seconds)
            await asyncio.sleep(10)

    except asyncio.CancelledError:
        logger.info("VSOC Monitoring System task cancelled.")
    except Exception as e:
        logger.error(f"Unexpected error in monitoring loop: {e}")

def main():
    logger = setup_logging()
    try:
        asyncio.run(monitor_loop(logger))
    except KeyboardInterrupt:
        logger.info("VSOC Monitoring System stopped by user.")

if __name__ == "__main__":
    main()
```

7.4 Threat_intelligence.py

```
def update_threat_intelligence():
    try:
        # Simulate updating threat intelligence feeds from external
sources
        print("Threat intelligence feeds updated.")
    except Exception as e:
        print(f"Error updating threat intelligence: {e}")
```

8. System Testing

The purpose of testing the Virtual Security Operation Center (VSOC) is to ensure that all modules—such as log collection, threat detection, incident response, alerting, and threat intelligence—work reliably and cohesively to detect and respond to cybersecurity threats. It validates the correctness, performance, integration, and security of the system under various operational scenarios.

Testing helps identify defects and ensures the system performs as expected, enabling early detection of vulnerabilities and improving the overall quality and resilience of the security framework.

8.1 Unit Testing

8.1.1 Log Testing and Parsing

Objective: Test the asynchronous log collection function to ensure it returns structured data.

- Ensure logs are collected without delay or corruption.
- Test with empty, malformed, or unexpected log formats.

8.1.2 Threat Detection Logic

Objective: Validate analyze_logs() function identifies known threats in sample logs.

- Test for correct pattern matching (e.g., using keywords like "unauthorized", "threat").
- Confirm that non-threat logs are ignored.

8.1.3 Alerting and Response

Objective: Ensure send_alert() and respond_to_threat() handle inputs correctly and generate appropriate outputs.

- Test for system response when multiple threats are sent simultaneously.
- Verify console output simulates real-world alerts.

8.2 Integration Testing

8.2.1 Module Interaction

Objective: Test seamless integration between log management, incident response, and alerting.

- Validate that threat detection triggers both response and alert modules.
- Confirm that state updates reflect correctly across modules.

8.2.2 Shared State Management

Objective: Test the correct population of shared_state dictionaries used by the dashboard and API.

- Ensure real-time updates to logs, threats, and alerts arrays.
- Simulate missing or corrupted data to test system recovery.

8.3 Component Testing

8.3.1 Dashboard API

Objective: Ensure /api/status returns accurate JSON data for frontend.

- Check response format and data freshness.
- Simulate API failure and confirm fallback behavior.

8.3.2 Threat Intelligence Updates

Objective: Verify the periodic execution of update_threat_intelligence().

- Ensure this doesn't block the monitoring loop.
- Validate log messages indicating update success.

8.4 Functional Testing

8.4.1 Monitoring Workflow

Objective: Simulate full monitoring loop from log collection to dashboard output.

Include normal, threat, and error conditions.

Confirm detection, response, and alert cycles.

8.4.2 Error Handling and User Feedback

Objective: Simulate module failures (e.g., crash in log collection) and ensure system logs appropriate errors.

9. Test Cases

S No	Test Case	Expected Result	Result	Remarks
1	Test for log collection functionality	Log entries are successfully collected from simulated sources.	Collected	Logs collected as expected using collect_logs().
2	Detect threats from log analysis	Logs containing keywords like "threat" or "unauthorized" are flagged.	Detected	Threats successfully identified using analyze_logs().
3	Test incident response execution	Response message printed for each detected threat.	Responded	Incident response simulated via console output.
4	Verify alert generation mechanism	Alert message printed for every threat detected.	Alerted	Alerts triggered as expected using send_alert().
5	Check shared state updates	shared_state reflects latest logs, threats, and alerts.	Updated	Shared state correctly updated and served to the dashboard.
6	Validate dashboard display	Dashboard auto-refreshes and displays accurate logs, threats, and alerts.	Displayed	Dashboard reflects real-time system status.
7	Test API endpoint /api/status	Returns valid JSON with logs, threats, and alerts.	Returned	JSON response verified in browser and API tools.

8	Simulate threat intelligence update	Console logs show updated threat intelligence messages.	Updated	Threat intelligence Module executed successfully.
9	Check log file writing and rotation	Logs are written to vsoc.log, and old logs rotate when size limit is reached.	Rotated	The logging system works as intended with rotation.
10	Simulate error in log collection	Error is logged and the system continues operation.	Handled	System error handled gracefully and logged.

Table - 9.1 Test Cases

10. OUTPUTS

1) When you start the VSOC system by running python dashboard.py, the backend monitoring begins, and the Flask web server launches, serving the dashboard. Initially, the dashboard displays a clean interface with three sections labeled Logs, Threats Detected, and Alerts, each showing "No logs available," "No threats detected," and "No alerts," respectively.

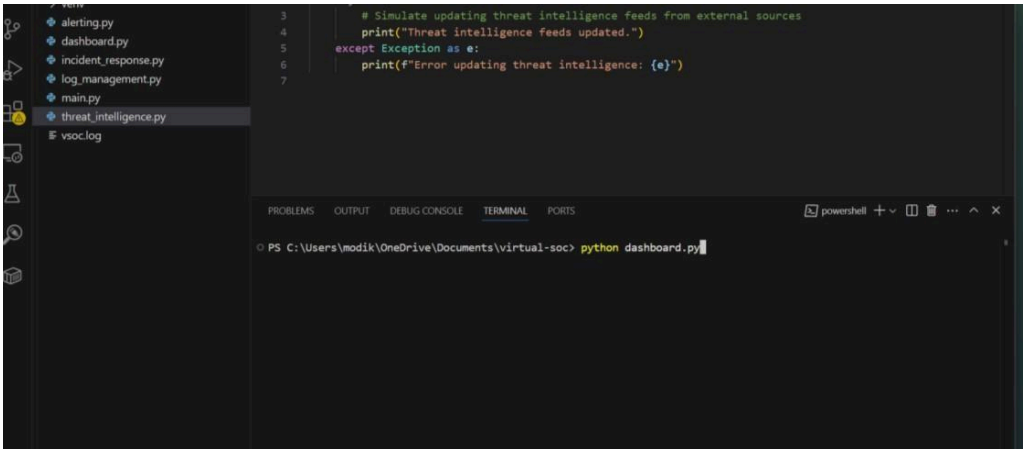


Figure-10.1 Run the code

2) As you generate or append logs in the logs directory, for example, by adding entries like "User login succeeded" and "Threat detected: suspicious login attempt," the dashboard automatically updates within a few seconds. The Logs section now shows the new entries, while the Threats Detected section highlights the suspicious login attempt. The Alerts section lists the automated response action triggered by the system, indicating that a threat was responded to.

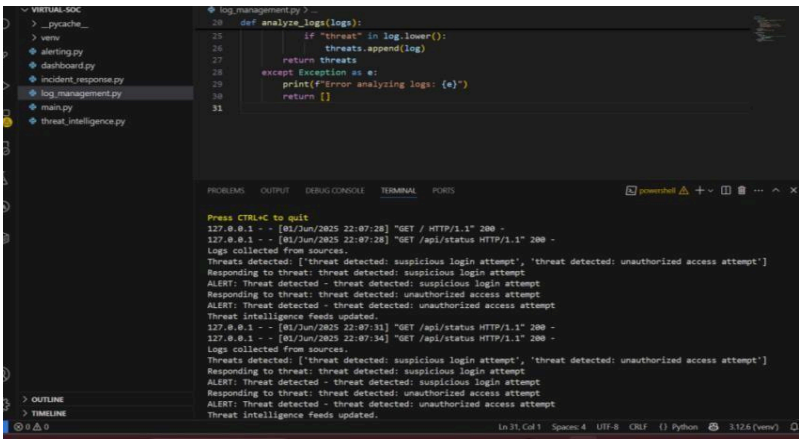


Figure-10.2 Adding Logs

3) If additional suspicious activities are logged, such as "Unauthorized access attempt" and "Malware signature detected in file scan," the dashboard continues to grow. The Logs section displays all the entries, while the Threats Detected section lists multiple threats identified by the system. The Alerts section reflects the corresponding automated responses for each detected threat.

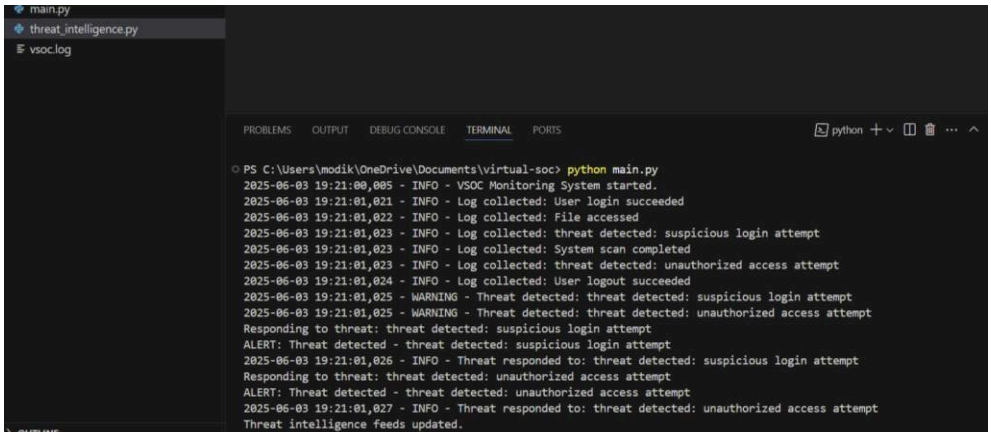


Figure-10.3 Server Started

4) In above screen python server started and now open the browser and enter the URL as http://127.0.0.1:5000/ and press enter key to get the page below.

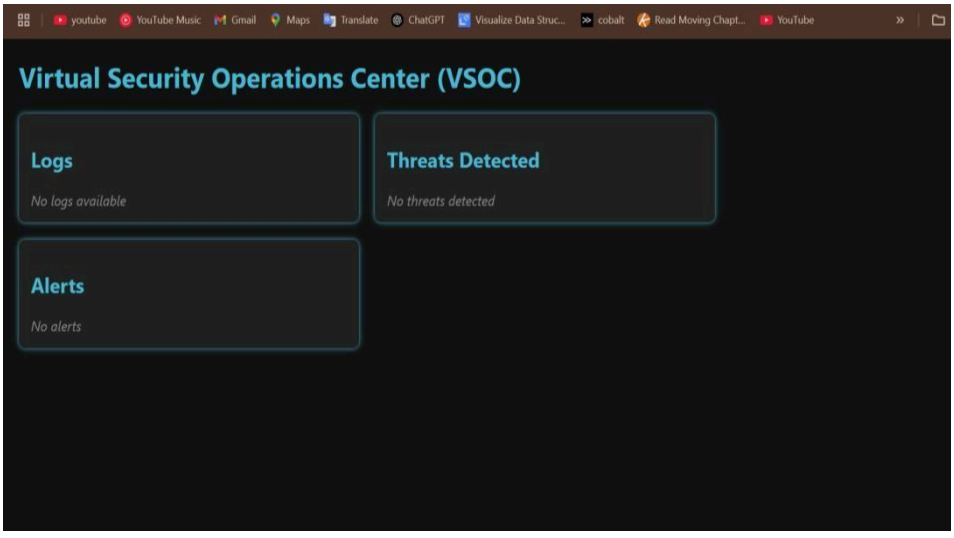


Figure-10.4 Home Page

5) Periodically, the threat intelligence module updates its feed, enhancing the system's detection capabilities. This update is logged in the console, ensuring that the system remains current with

the latest threat information.



Figure-10.5 New Logs

6) It sends alerts of any suspicious activity about the user, IP address and description of the activity.

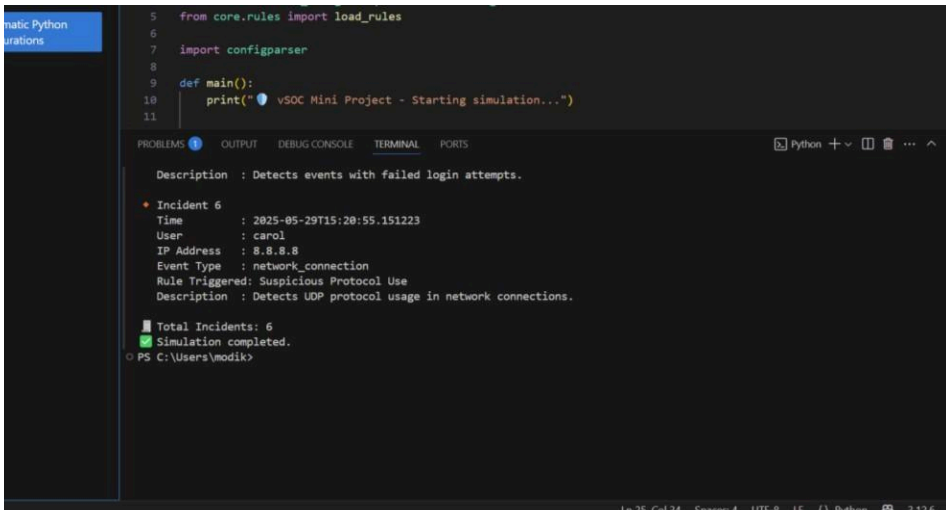


Figure-13 Alert

7) Throughout the operation, the system continuously monitors logs, and new entries appear live on the dashboard. The interface remains responsive, allowing you to scroll through logs, view details, and track trends in real-time.

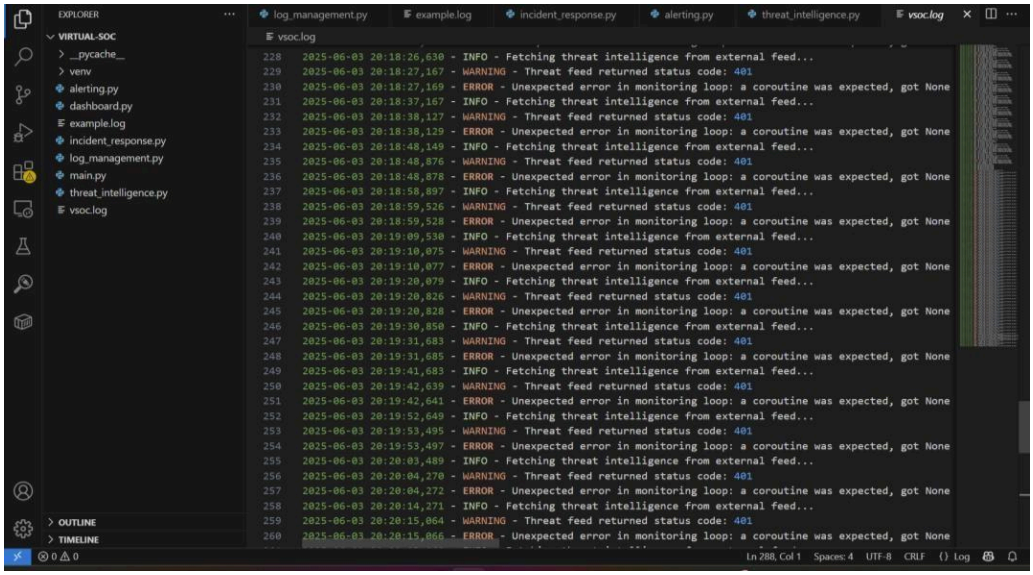


Figure-14 Storing logs

8) The dashboard provides a dynamic and interactive experience, helping you stay informed about potential security threats and the system's status.



Figure-15 Dashboard showing all the activity

11. Conclusion

The Virtual Security Operation Center (vSOC) mini project successfully demonstrates the fundamental concepts of modern cybersecurity monitoring and threat detection using automation and intelligence. By simulating event generation, log analysis, and incident reporting, the system showcases how organizations can proactively manage potential threats in real-time.

The integration of components such as event simulators, incident managers, rule-based analysis, and reporting mechanisms reflects the essential building blocks of an actual Security Operations Center. Furthermore, the incorporation of AI/ML-based threat detection (as visualized in the architecture) highlights the potential for future expansion into more advanced and intelligent security systems.

This project not only improves understanding of how a vSOC operates but also emphasizes the importance of automation in cybersecurity. It can serve as a foundation for scaling up into more complex security infrastructures involving live data, SIEM tools, and real-time response frameworks.

In conclusion, this mini project reinforces the significance of proactive threat detection and provides a simplified yet practical model of how digital security operations can be managed effectively in virtual environments.

12. FUTURE ENHANCEMENTS

Future enhancements for the Virtual Security Operations Center (vSOC) project could include integrating with threat intelligence platforms for proactive detection of known threats and deploying machine learning models for anomaly and zero-day attack detection. Incorporating SOAR tools would automate incident response workflows, significantly reducing manual intervention. Adding real-time dashboards and visual analytics using Grafana or Kibana would enhance monitoring efficiency. Support for multi-cloud and hybrid environments can provide unified visibility, while deploying the vSOC in a containerized architecture with Kubernetes ensures scalability and ease of deployment. Additionally, integrating User and Entity Behavior Analytics (UEBA) would help detect insider threats, and implementing long-term log archiving ensures compliance with regulatory requirements. Simulated attack testing environments using tools like Caldera can also validate and strengthen the SOC's detection and response capabilities.

13. References

- Ali, M.et al. (2021). A Cloud-Based Virtual SOC for Scalable Cybersecurity Operations. IEEE Xplore from https://cybersecurityguide.org/careers/cyber-crime-investigator/

- Raj, R., & Kumar, S. (2020). Implementation of Virtual SOC using SIEM in Hybrid Cloud. IEEE Xplore.

- Smith, A. (2019). Virtual SOC: Bridging Gaps in Cybersecurity for SMEs. MDPI.

- Lee, J., & Park, K. (2022). Designing a Virtual Security Operations Center with SOAR. Journal of Cybersecurity.

- Patel, R., et al. (2021). Cloud-Native Security Operations Center Architecture. ACM Computing Surveys.

- Chen, L., & Zhang, Y. (2020). An Open Source Framework for Virtual SOC Deployment. IEEE Software https://www.sans.org/cyber-security-courses/cybercrime-intelligence/

- Kumar, S., & Gupta, P. (2023). Machine Learning for Anomaly Detection in Virtual SOCs. Elsevier Journal of Information Security.

- Williams, D., & Johnson, T. (2022). Evaluation of Virtual SOC Performance in Hybrid Cloud Environments. International Journal of Network Security.

- SANS Institute. (n.d.). Security operations center (SOC) – The basics. SANS. Retrieved May 29, 2025, from https://www.sans.org/security-resources/security- operations-center/

- Kumar, R., & Chilamkurti, N. (2020). Automated incident response systems: A review. IEEE Access, 8, 130618–130634 from https://doi.org/10.1109/ACCESS.2020.3002237